Zpracování skriptů SQL

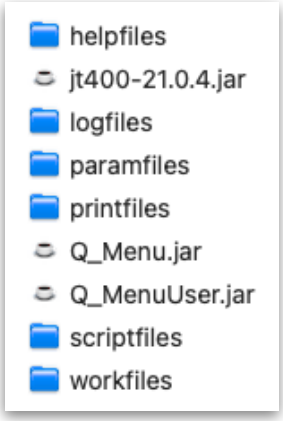
Řešitelská dokumentace

Obsah

Obsah 2
Adresář aplikace 3
Vývojový adresář 4
Seznam aplikačních programů 6
Programy v balíku queries – obsahují všechny funkce 6
Výpočetní programy 6
Lokalizační programy 6
Popis činnosti aplikačních programů 7
Třída Q_Menu 8
Třída Q_MenuUser 9
Q_ScriptEditCall – údržba skriptů SQL 11
Q_ScriptRunCall – údržba a provoz skriptů 12
Provedení skriptu – performScript() 12
Provedení SQL příkazu – performSqlStatement() 13
Pomocné metody 13
Q_ScriptRun – spuštění příkazu SQL 14
Zahájení výpočtu 14
Příkaz s parametry 14
Příkaz bez parametrů 15
Zpracování skriptu typu Update 15
Zpracování skriptu typu Query (dotazu) 15
Zpracování prvního řádku výsledné tabulky (result set) 15
Zpracování dalších řádků výsledné tabulky v cyklu 17
Závěr zpracování výsledné tabulky 18
Metoda buildColumnHeaders() - Sestavení hlaviček sloupců 18
Metoda editNumericDecimal() - formátování dekadických čísel 18
Metoda processLevelBreaks() - zpracování skupinových úrovní (L1, ...) 19
Metoda processLastLevelBreak() - zpracování všech úrovní na konci výsledné tabulky 20
Metoda initializeAccumulators() - inicializace úrovňových i koncových střádačů. 20
Obsluha zpráv (informačních a chybových) 20
Q_PromptParameters – zadání hodnot pro parametry příkazů SQL 21
Q_PrintOneFile – zobrazení a tisk výsledku dotazu 22
Zobrazení výsledku skriptu 22
Tisk výsledku skriptu na papír 23
Zápis výsledku skriptu do souboru 24
Q_PromptAddScript – zadání jména k vytvoření skriptu 24
Q_PromptImportScript – zadání jména skriptu k importu z adresáře IFS 25
Q_ImportOneFromAS400 – import vybraného skriptu z IFS 25
Q_ExportOneToAS400 – export vybraného skriptu do IFS 26
Q_ImportAllFromAS400 – import všech skriptů z IFS 26
Q_ExportAllToAS400 – export všech skriptů do IFS 27
Q_ConnectDB – spojení JDBC se serverem 27
Q_ScriptEdit – editor skriptu 28
Q_ColumnLists – sestavení seznamu sloupců 29

Adresář aplikace

Aplikace je uložena v jednom adresáři a skládá se z následujících objektů podle obrázku.

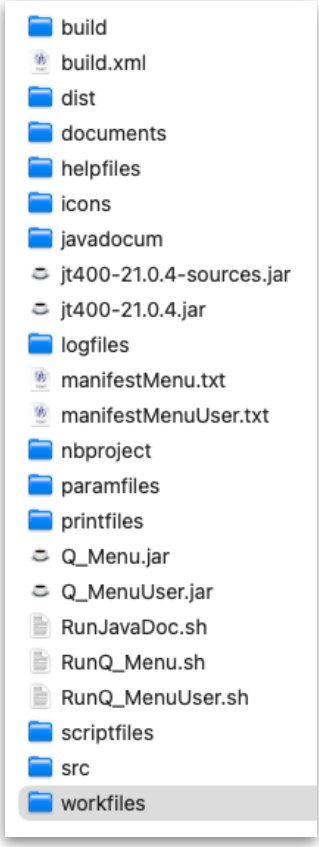


- `helpfiles` je adresář obshující uživatelskou dokumentaci pro Help.
- `jt400-21.0.4.jar` je soubor podpůrných programů ze soustavy *IBM i Toolbox for Java*.
- `logfiles` je adresář obsahující soubory `err.txt` a `out.txt`, do nichž se zapisují chybové zprávy a ladicí tisky `System.err` a `System.out`.
- `paramfiles` je adresář obsahující jediný soubor `Q_Parameters.txt` s parametry aplikace.
- `printfiles` je adresář obsahující textové soubory s výsledky skriptů.
- `Q_Menu.jar` je hlavní soubor programů, který dvojitým klepnutím spouští program *Q_Menu* otevírající okno s nabídkou funkcí pro správce aplikace.
- `Q_MenuUser.jar` je soubor programů, který dvojitým klepnutím spouští program *Q_Menu* otevírající okno s nabídkou funkcí pro běžné uživatele aplikace.
- `scriptfiles` je adresář obsahující textové soubory se skripty. Soubory mají koncovku `.sql`.
- `workfiles` je adresář obsahující textový soubor `Print.txt` s výsledkem naposled spuštěného skriptu. Může obsahovat libovolné další objekty podle volby uživatele.

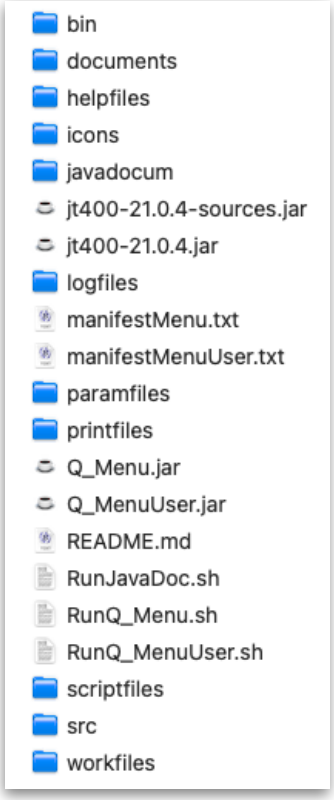
Vývojový adresář

K vývoji a údržbě je určen adresář projektu obsahující shora uvedené objekty aplikace a navíc objekty sloužící k vývoji projektu.

V prostředí NetBeans



V prostředí Eclipse



Vývojový adresář obsahuje navíc tyto objekty:

- *build/classes* (NetBeans) nebo *bin* (Eclipse) - adresář se soubory *.class* aplikačních programů v balících queries, queries.user a locales
- *src* - adresář se soubory *.java* zdrojových aplikačních programů v balících (adresářích) queries, queries.user a locales
- *RunQ_Menu.sh* - shell skript (NetBeans) k vytvoření souboru `Q_Menu.jar`

```
echo ==========================================================================
echo --- Create executable Jar file Q_Menu.jar
echo ==========================================================================
echo The following command gets the directory in which the script is placed
script_dir=$(dirname $0)
echo script_dir=$(dirname $0)
echo $script_dir
echo ----------------------------------------------------------
echo The following command makes the application directory current
cd $script_dir
echo cd $script_dir
echo ----------------------------------------------------------
echo The following command creates the Jar file in the current directory
echo jar cvfm  Q_Menu.jar  manifestMenu.txt  -C build/classes queries/Q_Menu.class  -C build/classes queries -C build/
classes locales
jar cvfm  Q_Menu.jar  manifestMenu.txt  -C build/classes queries/Q_Menu.class  -C build/classes queries -C build/
classes locales
echo ----------------------------------------
```

```
echo The following command executes the Jar file
echo java -jar Q_Menu.jar
java -jar Q_Menu.jar
```

- *manifestMenu.txt* - podpůrný soubor ke skriptu *RunQ_Menu.sh*

```
Manifest-Version: 1.0
Class-Path: ./jt400Small.jar
Main-Class: queries/Q_Menu
```

- *RunQ_MenuUser.sh* - shell skript (NetBeans) k vytvoření souboru *Q_MenuUser.jar*

```
echo ==============================================================================
echo --- Create executable Jar file Q_MenuUser.jar
echo ==============================================================================
echo The following command gets the directory in which the script is placed
script_dir=$(dirname $0)
echo script_dir=$(dirname $0)
echo $script_dir
echo ------------------------------------------------------------
echo The following command makes the application directory current
cd $script_dir
echo cd $script_dir
echo ---------------------------------------------------------------
echo The following command creates the Jar file in the current directory

echo jar cvfm  Q_MenuUser.jar  manifestMenuUser.txt  -C build/classes  queries.user/Q_Menu.class  -C build/classes
queries -C build/classes  locales
jar cvfm  Q_MenuUser.jar  manifestMenuUser.txt  -C build/classes  queries.user/Q_Menu.class  -C build/classes  queries
-C build/classes  locales
echo -----------------------------------------
echo The following command executes the Jar file

echo java -jar Q_MenuUser.jar
java -jar Q_MenuUser.jar
```

- *manifestMenuUser.txt* - podpůrný soubor ke skriptu *RunQ_MenuUser.sh*

```
Manifest-Version: 1.0
Class-Path: ./jt400Small.jar
Main-Class: queries.user/Q_Menu
```

- *javadocum* - adresář s dokumentací programů aplikace
- *RunJavaDoc.sh* - shell skript pro Eclipse k vytvoření adresáře `javadocum`.

```
echo ==============================================================================
echo --- Create project documentation
echo ==============================================================================
echo The following command gets the directory in which the source folder is placed
script_dir=$(dirname $0)/src
echo script_dir=$(dirname $0)/src
echo $script_dir
echo ---------------------------------------------------------------
echo The following command makes the application source directory current
cd $script_dir
echo cd $script_dir
echo ------------------------------------------------------------
echo The following command creates documentation in javadoc folder
echo javadoc -d ../javadocum queries locales
javadoc -d ../javadocum queries locales
```

Seznam aplikačních programů

Zdrojové aplikační programy jsou obsaženy v adresáři *src*. Přeložené programy jsou v adresáři *bin (Eclipse)* nebo *build.classes (NetBeans)*.

Programy v balíku `queries` – obsahují všechny funkce

Výpočetní programy

Q_Menu	Nabídka všech funkcí aplikace
Q_ScriptRunCall	Metody spouštění skriptu - *performScript()* a SQL příkazu - *performSqlStatement()*
Q_ScriptRun	Spuštění příkazu SQL (Query nebo non-Query)
Q_PromptParameters	Zadání proměnných hodnot pro parametry (otazníky) příkazů SQL
Q_PrintOneFile	Tisk výsledku dotazu
Q_ParametersEdit	Zadání a úprava parametrů aplikace
Q_ParametersCreate	Vytvoření parametrů aplikace, jestliže ještě neexistují
Q_Properties	Poskytnutí hodnot parametrů aplikace pro jednotlivé programy
Q_ScriptEditCall	Údržba skriptů - editace, export a import skriptů
Q_ScriptEdit	Editor skriptu
Q_PromptAddScript	Výzva k zadání jména k vytvoření nového skriptu
Q_PromptImpScript	Výzva k zadání jména skriptu k importu z adresáře IFS, když nebyl vybrán žádný řádek z tabulky
Q_ImportOneFromAS400	Import vybraného skriptu z IFS
Q_ExportOneToAS400	Export vybraného skriptu do IFS
Q_ImportAllFromAS400	Import všech skriptů z IFS
Q_ExportAllToAS400	Export všech skriptů do IFS
Q_ConnectDB	Spojení JDBC se serverem

Lokalizační programy

Lokalizační programy nejsou v pravém slova smyslu programy, ale jen seznamy textů v uspořádání (klíč, hodnota). Slouží ostatním programům k začlenění textů uvedením klíče.

L_ButtonBundle_cs_CZ	Nápisy tlačítek v češtině
L_ButtonBundle_en_US	Nápisy tlačítek v angličtině
L_MessageBundle_cs_CZ	Zprávy v češtině
L_MessageBundle_en_US	Zprávy v angličtině
L_TitleLabelBundle_cs_CZ	Nadpisy, návody a doprovodné texty v češtině
L_TitleLabelBundle_en_US	Nadpisy, návody a doprovodné texty v angličtině

Popis činnosti aplikačních programů

Vysvětluje se činnost programů z hlediska jejich celkové logiky a návaznosti.

Existují dvě hlavní třídy ke spuštění aplikace. Jedna je *Q_Menu* určená pro správce aplikace a druhá je *Q_MenuUser* určená pro běžného uživatele. Obě mají hlavní metody main(String... strings). Obě volají metodu *createQ_Menu(boolean fullMenu)*. Argument ve volání je buď *true* v první nebo *false* v druhé. Argument *fullMenu* se pak v metodě *createQ_Menu()* testuje a podle jeho hodnoty *true* nebo *false* se konstruuje tvar okna a do výsledného programu se zařazují objekty pro správce nebo pro uživatele.

Třída Q_Menu

Hlavní metoda *main()* je vyvolána s argumentem, který není využit, sama vyvolá konstruktor třídy a zobrazení prvního okna metodou *createQ_Menu(**true**)*. Hodnota *true* znamená, že se tvoří prostředí pro správce aplikace.

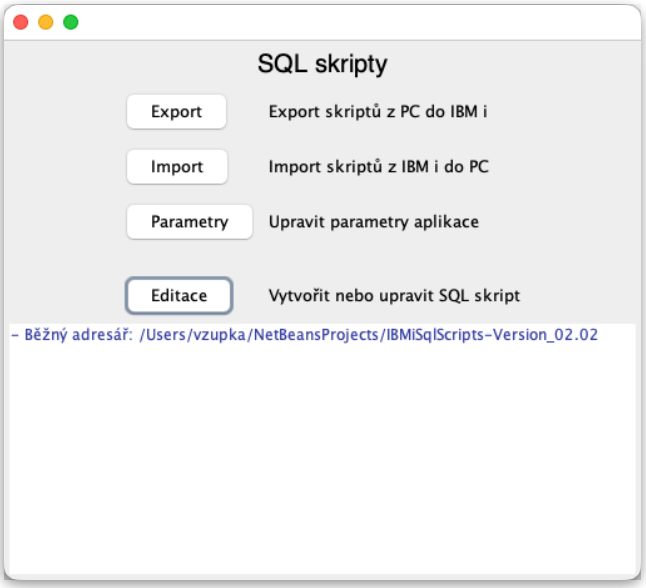
```
Q_Menu cmn = new Q_Menu();
cmn.createQ_Menu(true);
```

Metoda *createQ_Menu()* zjistí uživatelský adresář "*user.dir*" a v něm vytvoří adresáře *paramfiles*, *logfiles*, *scriptfiles*, *printfiles*, *workfiles*.

Dále vytvoří soubory *err.txt* a *out.txt* v adresáři *logfiles* a *přesměruje* do nich systémové soubory Javy *System.err* a *System.out*.

Metoda pak přečte hodnoty parametrů aplikace podle jejich jmen ze souboru *Q_Parameters.txt* prostřednictvím programu *Q_Properties*. Jestliže v adresáři *paramfiles* soubor *Q_Parameters.txt* ještě neexistuje, vytvoří počáteční hodnoty parametrů aplikace programem *Q_ParametersCreate. main()*.

Pak vytvoří okno, z nějž správce aplikace vyvolává další programy prostřednictvím tlačítek:



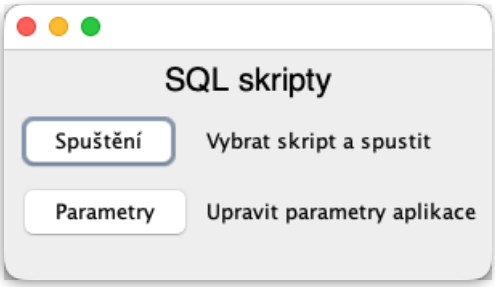
1. **toAS400**Button.addActionListener(sb -> {...});
   Tlačítko Export. Volá *Q_ExportAllToAS400* metodou *transferAllToAS400()* - Export všech skriptů do IFS
2. **fromAS400**Button.addActionListener(sb -> {...});
   Tlačítko Import. Volá *Q_ImportAllFromAS400* metodou *transferAllFromAS400()* - Import všech skriptů z IFS
3. **parameters**Button.addActionListener(mb -> {...});
   Tlačítko Parametry. Volá *Q_ParametersEdit*(***true***) metodou *main(null)* - Zadání a úprava parametrů aplikace
4. **editScript**Button.addActionListener(tb -> {...});
   Tlačítko Editace. Volá *Q_ScriptEditCall* metodou *buildScriptList()* - Vytváření, údržba a provádění skriptů

Třída Q_MenuUser

Hlavní metoda *main()* je vyvolána s argumentem, který není využit, sama vyvolá konstruktor třídy *Q_Menu* a provede metodu *createQ_Menu(**false**)*. z

```
Q_Menu cmn = new Q_Menu();
cmn.createQ_Menu(false);
```

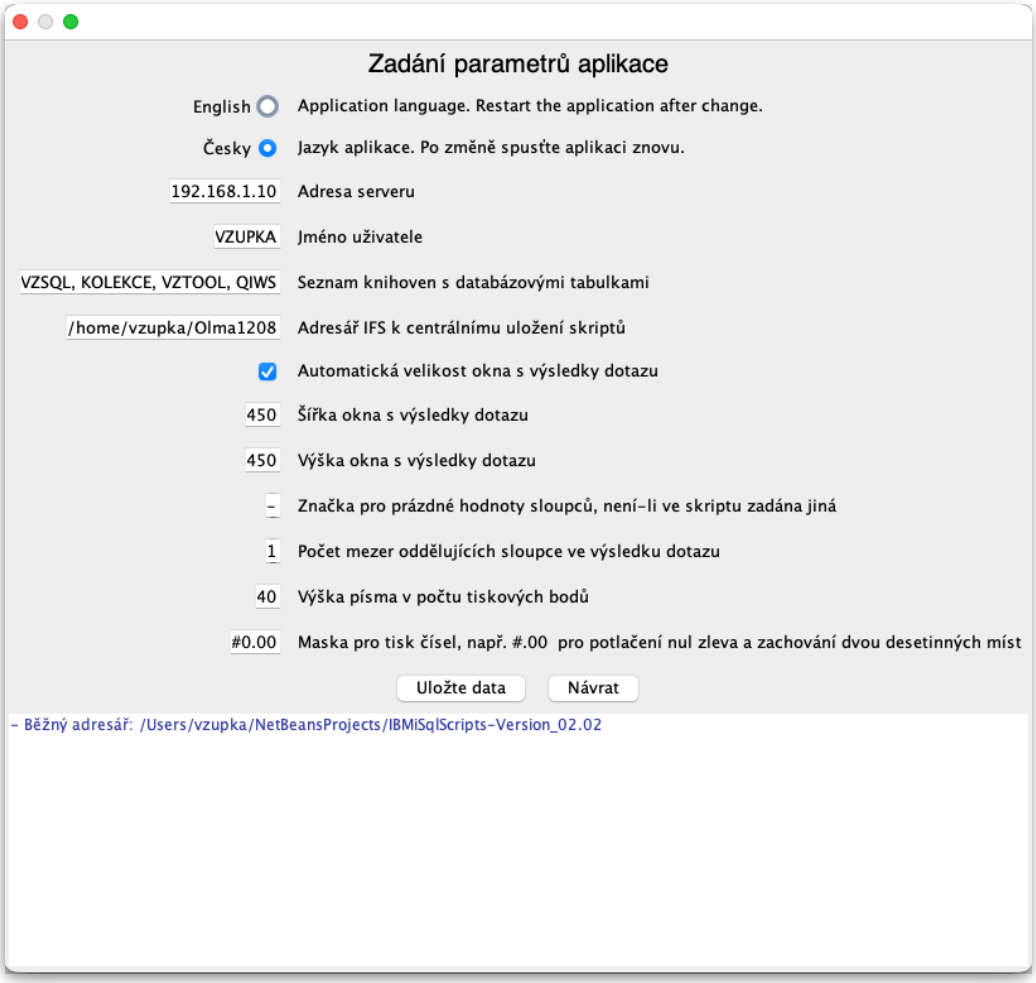
Tím zobrazí první okno aplikace pro běžného uživatele.



Má dvě tlačítka, *Spuštění* a *Parametry*.

1. **selectQuery**Button.addActionListener(tb -> {...});
   Tlačítko Spuštění. Volá *Q_ScriptEditCall(**false**)* metodou *buildScriptList()* - Provádění skriptů
2. **parameters**Button.addActionListener(mb -> {...});
   Tlačítko Parametry. Volá *Q_ParametersEdit(false)* - Zadání a úprava parametrů aplikace pro uživatele

Dále se postupuje podle základní nabídky pro správce aplikace.



Q_ParametersEdit – úprava parametrů aplikace

Program je vyvolán z hlavní nabídky tlačítkem *Parametry* a vytvoří okno. Obsah okna tvoří dvě radiová tlačítka pro volbu jazyka aplikace, vstupní textová pole s doprovodnými popisy a tlačítko pro uložení vstupních hodnot. Jazyk aplikace a vstupní pole inicializuje počátečními hodnotami.

Zobrazená tlačítka *Uložte data* a *Návrat* pak umožní vykonávat funkcei (lambda):

saveButton.addActionListener(al -> {...});
Tlačítko Uložte data. Volá metodu *saveData()* pro uložení dat ve vstupních polích, radiových tlačítkách a zaškrtávacím políčku.
returnButton.addActionListener(al -> {...});
Tlačítko Návrat. Volá metodu *dispose()*, která zavře okno a vrátí se do hlavní nabídky.

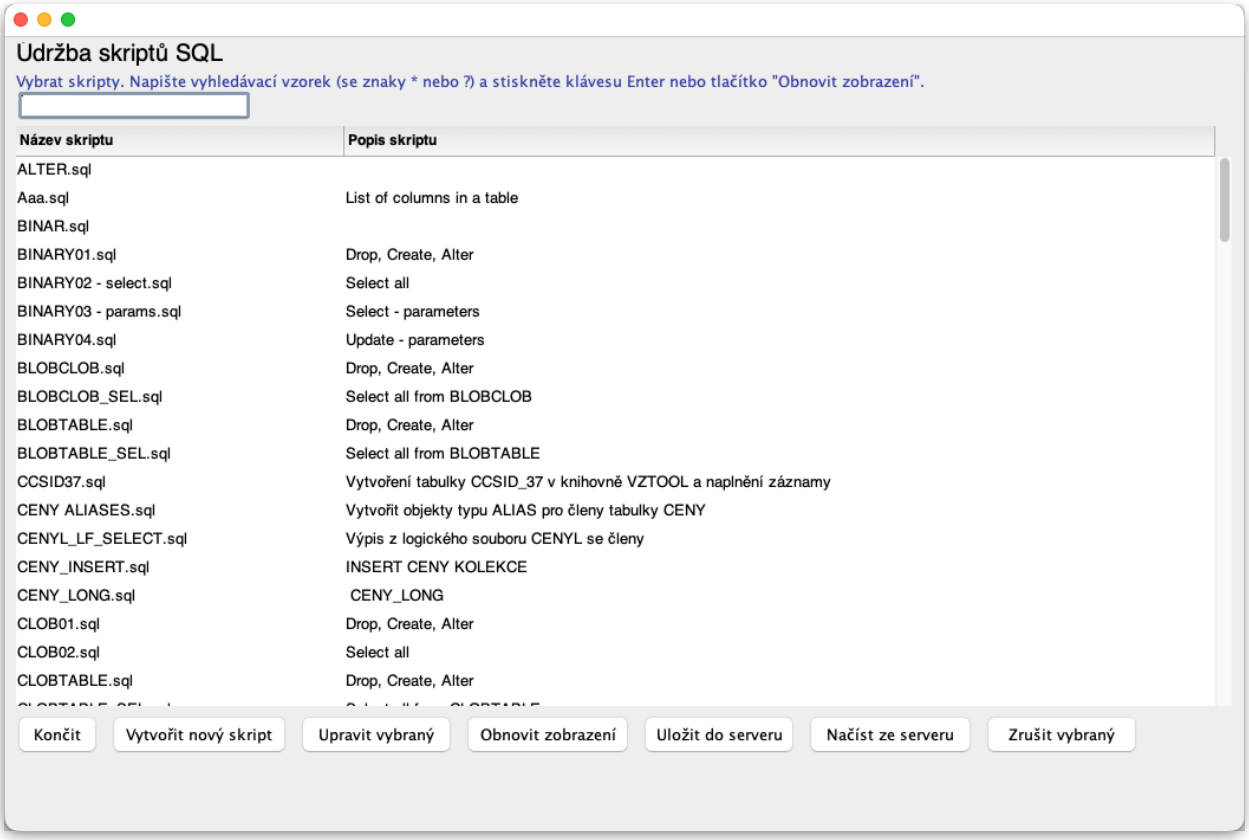
Uložení dat vykoná i stisk klávesy Enter, což je umožněno vnitřní třídou *SaveAction* a její metodou *actionPerformed()*, která volá metodu *saveData()*.

Metoda *saveData()* zkontroluje numerické parametry v textových polích, místo chybných dosadí 0 a znovu je zobrazí. Hodnoty textových polí, radiových tlačítek (jazyk aplikace) a zaškrtávacího políčka (automatická velikost okna) pak zapíše do souboru *Q_Parameters.txt* v adresáři “paramfiles”. Jména *properties* (s příklady) jsou:

```
AUTO_WINDOW_SIZE=Y
CARET=Long caret
COLUMN_SEPARATING_SPACES=1
DECIMAL_PATTERN=\#0.00
```

```
EDITOR_FONT=Monospaced
EDITOR_FONT_SIZE=16
FONT_SIZE=40
HIGHLIGHT=SQL
HIGHLIGHT_BLOCKS=SQL
HOST=192.168.1.10
IFS_DIRECTORY=/home/vzupka/Olma1208
LANGUAGE=cs-CZ
LIBRARY_LIST=VZSQL, KOLEKCE, VZTOOL, QIWS
MATCH_CASE=CASE_INSENSITIVE
NULL_PRINT_MARK=-
RESULT_WINDOW_HEIGHT=450
RESULT_WINDOW_WIDTH=450
SELECTION_MODE=Horizontal selection
USER_NAME=VZUPKA
```

Q_ScriptEditCall – údržba skriptů SQL



Program je volán z hlavní nabídky metodou *buildScriptList()*, která sestaví seznam skriptů ve formě tabulky, a to s pomocí metod *readInputFiles()* a *readLinesForScriptList()*.

Metoda *readInputFiles()* čte soubory s koncovkou `.sql` z adresáře “scriptfiles”. Jméno a popis skriptu zapíše do mapy *scriptNames*, kde klíčem je jméno souboru a hodnotou jeho popis. Popis skriptu zjistí z prvního textového řádku uvnitř přečteného souboru. Další řádky souboru nezkoumá. Je-li to komentář se dvěma pomlčkami, je popisem text za nimi. V opačném případě bude popis prázdný. Mapa pak slouží dalším metodám.

Metoda *readLinesForScriptList()* čte klíče a hodnoty z mapy *scriptNames* a zapisuje je do dvojrozměrného pole records[][]. Pak volá metodu *fillTable()*, která těmi hodnotami naplní řádky a sloupce tabulkového modelu (třída Q_TableModel).

Tlačítka pak umožňují vykonávat tyto funkce (lambda):

3. scriptList**Exit**Button.addActionListener(a -> {...});
   Smaže okno a vrací se do hlavní nabídky.
4. scriptList**Add**Button.addActionListener(a -> {...});
   Vyvolá dialog *Q_GetTextFromDialog()* k zadání jména souboru s příponou `.sql`.
5. scriptList**Upd**Button.addActionListener(a -> {...});
   Vyvolá pro vybraný skript editor *Q_ScriptEdit()*.
6. scriptList**Refresh**Button.addActionListener(a -> {...});
   Obnoví zobrazení tabulky skriptů.
7. scriptList**Export**Button.addActionListener(a -> {...});
   Exportuje vybraný skript metodou *transferOneToAS400()* programu *Q_ExportOneToAS400*.
8. scriptList**Import**Button.addActionListener(a -> {...});
   Importuje skript ze serveru metodou *importScript(),* byl-li vybrán řádek, nebo volá metodu *runPrompt()* programu *Q_PromptForScriptName*, která nabídne zadání jména skriptu (nebyl-li vybrán řádek) a import takto zadaného skriptu.
9. scriptList**Del**Button.addActionListener(a -> {...});
   Odstraní vybraný skript z tabulky.

Q_ScriptRunCall – údržba a provoz skriptů

Je volán z programu *Q_ScriptEdit*. Spouští vybraný skript metodou *performScript()* a získá návratový kód *retCode* (typu String[]). Jeho první prvek retCode[0] je příznak zda byl skript spuštěn bez proměnných parametrů (text “runScript”) nebo s parametry (“runPrompt”). Druhý prvek retCode[1] je text zprávy. Není-li vybrán žádný řádek, ohlásí chybu.

Provedení skriptu – performScript()

Vytvoří objekt spojení *conn* s databázovým serverem voláním metody Q_ConnectDB#*connect()* a připraví lokalizovaný formát současného data nebo připraví chybovou zprávu do proměnné retCode.

Vytvoří textové soubory (Print.txt a *jméno-skriptu*.txt) s nadpisem skriptu (popis a datum). Čte text skriptu ze vstupního souboru, analyzuje jej. Průběžně střádá komentářové příkazy do seznamů předávaných dále jako parametry volání metody *Q_ScriptRun#runScript()*. Ta se volá přímo nebo prostřednictvím metody *Q_PromptParameters#runPrompt()*. Seznamy potřebuje metoda *runScript()* k provedení a vyhodnocení skriptu.

```
static ArrayList<String[]> markerArrayList = new ArrayList<>();
final String PARAM_PREFIX = "--;?";

static ArrayList<String[]> headerArrayList = new ArrayList<>();
final String HEADER_PREFIX = "--;H";

static ArrayList<String[]> totalArrayList = new ArrayList<>();
final String TOTAL_PREFIX = "--;T";

static ArrayList<String[]> patternArrayList = new ArrayList<>();
final String DECIMAL_PREFIX = "--;D";

static ArrayList<String[]> printArrayList = new ArrayList<>();
final String PRINT_PREFIX = "--;P";

static ArrayList<String> titleArrayList = new ArrayList<>();
final String TITLE_PREFIX = "--;t";

static ArrayList<String[]> levelArrayList = new ArrayList<>();
```

```
final String LEVEL_PREFIX = "--;L";

static ArrayList<String[]> summaryArrayList = new ArrayList<>();
final String SUMMARY_PREFIX = "--;S";

static ArrayList<String[]> summaryIndArrayList = new ArrayList<>();
final String SUM_IND_PREFIX = "--;s";

static ArrayList<String[]> omitArrayList = new ArrayList<>();
final String OMIT_PREFIX = "--;O";
```

Při analýze dělí skript na jednotlivé SQL příkazy a odstraňuje oddělující středníky. Pro každý SQL příkaz vyvolá k jeho provedení metodu

```
retCode = performSqlStatement(connection, scriptDescription);
```

Jestliže to byl příkaz nedotazového typu, zvýší čítač takových příkazů. Informaci o tom, zda to byl příkaz nedotazového typu ("UPDATE"), získá z položky *retCode[0]* návratového kódu.

Byl-li zpracován alespoň jeden nedotazový příkaz (typu "UPDATE"), přečte soubor *Print.txt* (celý skript a ještě text případných výsledků dotazů) a *zobrazí* jej v okně programu Q_PrintOneFile, kterému předá *nulový* počet hlavičkových řádků a headerArrayList = *null*. Nedotazový příkaz totiž nemá žádné sloupcové hlavičky. Při formátování stránek se tak na začátek každé stránky nebude zařazovat nic (na rozdíl od jednotlivého dotazového příkazu).

Provedení SQL příkazu – performSqlStatement()

Metoda *performSqlStatement()* provede SQL příkaz a vrátí návratový kód retCode.

Zjistí popis příkazu jako text za dvěma pomlčkami "--" začínajícími v 1. pozici prvního takového řádku (není-li to text "--;"). Nenajde-li takový řádek, dosadí prázdný popis.

Nejsou-li definovány proměnné parametry, spustí metodu Q_ScriptRun#runScript() s potřebnými argumenty (viz dále) a *připíše* k retCode[0] text "runScript", (ponechá tedy předchozí text "QUERY" nebo "UPDATE").

Jsou-li definovány proměnné parametry, zkontroluje počet údajů v každém definičním řádku. Je-li jich jiný počet než 4, začáteční zadané převezme a další doplní prázdným řetězcem. Jinak volá metodu *runPrompt()* programu *Q_PromptParameters()* s potřebnými argumenty (jako pro runScript()).

<u>Poznámka:</u> Metoda *runPrompt()* dále volá metodu *runScript()* programu *Q_ScriptRun* a *připíše* k retCode[0] text "runPrompt", (ponechá tedy předchozí text "QUERY" nebo "UPDATE").

Pomocné metody

Metoda *getMarkerValues()* extrahuje hodnoty z jednoho řádku definičního příkazu --;?.
Metoda *getHeaderValues()* extrahuje hodnoty z jednoho řádku definičního příkazu --;H.
Metoda *getTotalValues()* extrahuje hodnoty z jediného (prvního) řádku definičního příkazu --;T.
Metoda *getPatternValues()* extrahuje hodnoty z jednoho řádku definičního příkazu --;D.
Metoda *getPrintValues()* extrahuje hodnoty z jediného (prvního) řádku definičního příkazu --;P.
Metoda *getOmitValues()* extrahuje hodnoty z jednoho řádku definičního příkazu --;O.
Metoda *getTitleValues()* extrahuje hodnoty z jednoho řádku definičního příkazu --;t.
Metoda *getLevelValues()* extrahuje hodnoty z jednoho řádku definičního příkazu --;L.
Metoda *getSummaryValues()* extrahuje hodnoty z jednoho řádku definičního příkazu --;S.

Metoda *getSummaryIndValues()* extrahuje hodnoty z jednoho řádku definičního příkazu --;s.
Vnitřní třída *Q_TableModel* zařizuje plnění a chování tabulky *JTable scriptList*.

Q_ScriptRun – spuštění příkazu SQL

Tento program je ústředním místem aplikace, protože spouští SQL příkaz a vyhodnocuje jej. Nemá okno, je volán z programů *Q_ScriptRunCall* a *Q_PromptParameters* metodou ***runScript()*** s několika parametry.

```
public String[] runScript(Connection connection, String scriptName, String stmtDescription,
    String statement, ArrayList<String[]> markerArrayLst, ArrayList<String[]> headerArrayLst,
    ArrayList<String[]> totalArrayLst, ArrayList<String[]> patternArrayLst,
    ArrayList<String[]> printArrayLst, ArrayList<String> titleArrayLst,
    ArrayList<String[]> levelArrayLst, ArrayList<String[]> summaryArrayLst,
    ArrayList<String[]> summaryIndArrayLst, ArrayList<String[]> omitArrayLst)
```

Parametry volání metody *runScript()* jsou

connection – objekt typu Connection pro JDBC,
scriptName – jméno skriptu,
stmtDescription – popis příkazu,
script – text příkazu,
markerArrayList – seznam čtveřic (pořadí, typ, text, hodnota),
headerArrayList – seznam n-tic textů (sloupcových hlaviček),
totalArrayList – posuvy před tiskem, po tisku, symbol prázdné hodnoty, skupinové sloupce,
patternArrayList – dvojice (maska, jméno sloupce),
printArrayList – formát tiskového výstupu,
titleArrayList – seznam titulních hlaviček s proměnnými
levelArrayList – seznam skupinových úrovní
summaryArrayList – seznam sumarizovaných sloupců
summaryIndArrayList – seznam textů k označení typu sumarizace (SUM, AVG, MAX, MIN, COUNT),
omitArrayList – seznam vynechávaných sloupců.

Program vrací návratový kód *retCode* (dvousložkové pole). První složka *retCode[0]* je informační text o výsledku, druhá složka *retCode[1]* je chybová zpráva nebo prázdný řetězec.

Zahájení výpočtu

Nejprve zkontroluje připojení k databázovému serveru s použitím třídy *Q_ConnectDB*. Není-li spojení se serverem, končí s chybovou zprávou.

Existuje-li spojení, pokračuje a vytvoří textovou oblast *resultTextArea* a cestu k tiskovému souboru "*jméno-skriptu.txt*".

Uloží seznam uživatelských hlaviček *headerArrayList* do pomocného seznamu *headerArrayListSaved* pro obnovení původních neupravených hlaviček po jejich úpravě (viz dále), i když je seznam prázdný. To pro případ, když se tato metoda - runScript() - znovu vyvolá z programu *Q_PromptParameters*.

Zpracuje případný seznam parametrů *markerArrayList*.

Příkaz s parametry

Není-li *markerArrayList* prázdný, připraví SQL příkaz

```
PreparedStatement pstmt = connection
. prepareCall(statement, ResultSet.TYPE_SCROLL_SENSITIVE, ResultSet.CONCUR_READ_ONLY);
```

a dosadí do něj pro každý parametr podle pořadí hodnoty rozlišené podle zadaných typů

```
pstmt.setString() – pro typ CHAR
pstmt.setInt() – pro typ INT, INTEGER
pstmt.setBigDecimal() – pro typ DEC, DECIMAL
pstmt.setDate() – pro typ DATE
pstmt.setTime() – pro typ TIME
pstmt.setTimestamp() – pro typ TIMESTAMP
pstmt.setObject() – pro všechny ostatní typy nebo není-li zadán typ
```

Provede připravený příkaz a rozhodne, zda příkaz je dotazový (Query) nebo jiný (Update):

```
boolean isQuery = pstmt.execute();
if (isQuery) {
  ResultSet rs = pstmt.getResultSet();
  query_or_update = "QUERY";
} else {
   query_or_update = "UPDATE";
}
```

Příkaz bez parametrů

Je-li *markerArrayList* prázdný, provede příkaz podle toho, zda jde o dotaz (Query) nebo ne (Update).

```
stmt = conn.createStatement();
boolean isQuery = stmt.execute(script);
if (isQuery) {
  rs = stmt.getResultSet();
  query_or_update = "QUERY";
} else {
  query_or_update = "UPDATE";
}
```

Poznámka: Dojde-li při provádění SQL příkazu k chybě, je chyba zachycena a zobrazí se spolu s příkazem v okně. Viz odstavec *Obsluha zpráv*.

Zpracování skriptu typu Update

Je-li typ skriptu Update, *připíše* “UPDATE” do návratového kódu retCode[0] a zprávu o úspěšném provedení příkazu zapíše do retCode[1]. Text příkazu zapíše do oblasti resultTextArea. Pak jej zapíše do obou textových souborů a za konec přidá text zprávu o úspěšném provedení příkazu.

Zpracování skriptu typu Query (dotazu)

- Je-li skript typu Query, *připíše* “QUERY” do návratového kódu retCode[0] a analyzuje výslednou tabulku *rs*.
- Do *resultTextArea* zapíše text popisu skriptu a datum.
- Zjistí charakteristiky *všech* datových sloupců v počtu znaků pomocí jejích *metadat* ResultSetMetaData rsmd = *rs.getMetaData()*;
- Zjistí případné *vynechávané* sloupce z řádků `--;0` a jejich jména zapíše do seznamu.

Zpracování prvního řádku výsledné tabulky (result set)

- Čte první řádek výsledné tabulky příkazem *rs.first()*. Je-li tabulka prázdná, končí s prázdným výsledkem. Není-li prázdná, pokračuje dále.
- Vypne příznak *isDataNull*.

- Zjistí charakteristiky *všech* sloupců výsledné tabulky z tabulky *rsmd* vytvořené dříve a uloží je do seznamů pro jména, indexy, typy, desetinná místa a délky. Má-li některý sloupec hodnotu NULL, zapne příznak *isDataNull*.
- Zjistí hodnoty a indexy *vynechávaných* sloupců. Pro jejich případné použití v titulních řádcích edituje hodnoty numerických typů podle masky (z řádku **`--;D`** nebo parametrů aplikace), hodnoty ostatních typu automaticky jako Object, prázdné hodnoty značkou *nullPrintMark*.
- Přečte jména a hodnoty *všech* sloupců *prvního řádku* (včetně vynechávaných) do seznamu detColNames a *detColValues* pro účely porovnání s dalšími řádky a pro dosazení do proměnných (**&sloupec**) v titulních řádcích **`--;t`**.
- Jsou-li zadány titulní řádky --;t, dosadí do nich hodnoty proměnných pomocí metody *getValuesForVariables()*.
- Zapíše titulní řádky do oblasti *resultTextArea*.
- Jsou-li zadány *vynechávané* sloupce **`--;O`**, zjistí *charakteristiky zbývajících sloupců* a uloží do seznamů colNames, colLengths, colTypes, colScales a colIndexes. Jinak se tam uloží charakteristiky *všech* sloupců zjištěné dříve, takže oba seznamy sloupců jsou totožné.
- Připraví seznam střádačů pro sumarizované sloupce z definic **`--;S`**.
  - Seznam přečtený ze skriptu setřídí podle správného pořadí podle seznamu sloupců v příkazu SELECT.
  - Pak vytvoří seznamy jmen střádaných sloupců summaryColNames a seznamy sumarizačních indikací summaryIndications pro další zpracování. Seznam indikací zaplní indikacemi **`S`**, **`A`**, **`M`**, **`m`**, **`C`** tak, aby byly v tomto pořadí a poznamená, zda daná indikace je alespoň jednou zadána (atLeastOneSUM, atLeastOneAVG, ...).
  - Je-li zadána sumarizace podle řídicích úrovní (**`L0`**, **`L1`**, ...), získá počáteční hodnoty řídicích sloupců pro porovnání na změnu použitím metody getFirstLevelBreakValues().
  - Inicializuje příznaky newPage(level) hodnotou false.
  - Je-li také zadán nějaký sumarizovaný sloupec, inicializuje střádače sumarizovaných sloupců (z řádků **`--;S`**) podle jednotlivých typů (dekadický, celočíselný, datum, čas, časové razítko, znakový). K tomu volá metodu initializeAccumulators().
- Vyhodnotí definice řádku **`--;T.`** Není-li *totalArrayList* prázdný, přečte obsah nultého prvku (prvního řádku --;T) a z něj čtyři údaje. První převede na číslo do proměnné *spaceB* (space before), druhý na číslo do *spaceA* (space after), třetí do řetězce *nullPrintMark*. Čtvrtý převede na počet mezer mezi sloupci a vytvoří řetězec *colSepSpaces* s udaným počtem mezer. Další údaje jsou jména sloupců, u nichž se sleduje změna hodnoty v souvislé skupině řádků.
- Sestaví sloupcové hlavičky uživatelské **`--;H`** nebo standardní a jejich maximální šířky, a to pomocí metody *buildColumnHeaders()*.
- Vytvoří (prázdné) seznamy startovních a koncových pozic sloupců, které budou naplněny dále.
- Sestaví úvodní texty (leading string) s označením typu funkce (SUM, AVG, MAX, MIN, COUNT nebo texty z řádku **`--;s`**) pro řádky se sumárními hodnotami. Přitom může vzniknout mezerový řetězec sloužící jako *předřazená výplň* v úvodním textu a v každém tiskovém řádku. To pouze v případě, že *první* sloupec v řádku je sumarizován.

- Sloupcové hlavičky (uživatelské --;H nebo standardní) dříve sestavené zařadí z upraveného seznamu headerArrayList do seznamu *columnHeaders* (pro tiskový program Q_PrintOneFile) a do bufferu *headerBuf*. Buffer pak přidá do oblasti resultTextArea pro výstup. Využije přitom *předřazené výplně* (i prázdné).
- Upraví startovní a koncové pozice sloupců s použitím *předřazené výplně* (i prázdné) podle maximální šířky.
- Vymaže řádky pro úvodní texty pro řádky se sumárními hodnotami: printLineSUM, printLineAVG, printLineMAX, printLineMIN, printLineCOUNT.
- Upraví seznam z řádků **`--;D`** (úprava dekadických čísel). Počet položek omezí na 2: je-li jich zadáno v řádku více, dosadí do nich prázdný řetězec.
- Jsou-li zadány sloupce v řádku **`--;T,`** zařadí jejich jména do seznamu levelColNames a prázdné řetězce do seznamu levelColValues jako výchozí hodnoty pro test na změnu.

Zpracování dalších řádků výsledné tabulky v cyklu

- Inicializuje výstupní buffer pro řádek (StringBuilder dataBuf) předřazenou výplní. Výplň je neprázdná jen tehdy, když se sumarizuje *první* sloupec).
- Je-li zapnut příznak *isDataNull* a není zadána žádná definice --;L, vloží do bufferu dataBuf stanovený počet prázdných řádků *spaceB*.
- Provede cyklus přes všechny (nevynechávané) sloupce:
  - Edituje hodnoty sloupců pomocí metody editCurrentRowColumnValue(), která používá metodu *editNumericDecimal()* pro dekadická čísla DECIMAL a NUMERIC, pro binární typy BINARY a VARBINARY převod do *hexadecimálních* znaků, pro typ NULL dosazení značky *nullPrintMark* a pro ostatní typy ponechá převod z obecného typu (rs.get*Object*()).
  - Zarovná editovaný údaj do šířky tiskového sloupce vpravo nebo vlevo podle typu pomocí metody *adjustColumn()*, která používá metodu *rightAdjust()* pro zarovnání vpravo a metodu *leftAdjust()* pro zarovnání vlevo. Vpravo se zarovnávají typy s desetinnými místy většími než nula a typy NUMERIC, DECIMAL, INTEGER, BIGINT, SMALLINT, ale bez typu TIMESTAMP (který má až 6 desetinných míst). Ostatní se zarovnávají vlevo.
  - Je-li zadán řádek **`--;T`**, zkontroluje u zadaných sloupců, zda se změnila jejich hodnota proti minulé (porovnávací) hodnotě. Když ano, nahradí ji mezerami. Když ne, nemění ji, ale dosadí hodnotu sloupce jako porovnávací hodnotu v seznamu levelColValues.
  - Zařadí upravený sloupec s přidáním mezisloupcových mezer jako další do řádkového bufferu *dataBuf*.
- Je-li zapnut příznak *isDataNull* a není zadána žádná definice --;L, vloží do bufferu dataBuf stanovený počet prázdných řádků *spaceA*.
- Vypne příznak *isDataNull*.
- Jsou-li zadány obojí definice **`--;L`** i **`--;S`**:
  - Zjistí indexy a typy sumarizovaných sloupců a uloží do seznamu summaryColIndexes a summaryColTypes.
  - Testuje změny řídicích úrovní L1, L2, ... a zpracuje je v metodě *processLevelBreaks()*.

  - Do oblasti *resultTextArea* se přitom zapisují řádky s úvodními texty a sumárními hodnotami podle zpracované změny úrovně.
  - Testuje příznak newPage pro všechny řídicí úrovně a je-li zapnut, zapíše na konec datového bufferu znak **`\u0001`**. To je znak binární jednička v Unicode. Byl zvolen proto, že je neviditelný. Ten pak vyhodnocuje tiskový program *Q_PrintOneFile* tak, že takový řádek tiskne až na další stránce (provede skok na novou stránku).
- Na konec bufferu zařadí znak nového řádku.
- Přidá buffer do oblasti *resultTextArea* pro výstup a vyčistí buffer pro nový řádek.
- Opakuje cyklus čtení řádků, dokud není konec výsledné tabulky.

Závěr zpracování výsledné tabulky

- Jsou-li zadány obojí řádky **`--;L`** i **`--;S`**, dokončí závěrečné sumarizování pro poslední změnu všech skupinových úrovní způsobenou koncem dat a je-li zadána úroveň **`L0`**, pak také koncovou sumarizaci (grand total). To provede metodou *processLastLevelBreak()*.
- Uzavře výslednou tabulku příkazem *rs.close()*.
- Zapíše výsledek dotazu, tj. obsah oblasti *resultTextArea* do textového souboru `Print.txt` v adresáři `workfiles` a do souboru v adresáři `printfiles` pod jménem z proměnné *scriptName* s koncovkou `.txt`.
- Spustí program Q_*PrintOneFile* voláním konstruktoru s parametry

```
new Q_PrintOneFile
    (scriptName, resultTextArea, nbrHdrLines, headerArrayList, printArrayList, columnHeaders);
```

  Ten zobrazí výsledky dotazu a umožní je vytisknout.

Metoda *buildColumnHeaders*() - Sestavení hlaviček sloupců

- Není-li *headerArrayList* prázdný, obsahuje hodnoty z komentářových řádků **`--;H`**. Program zjistí šířky všech hlaviček odpovídajících polohou datovým sloupcům a spočítá *maximum* z šířky každé *hlavičky* a datového *sloupce*. Pak pro každý definiční řádek každou hlavičku doplní mezerami do *maximální šířky* podle typu sloupce (čísla zleva, ostatní vpravo). Hlavičky, které chyběly v řádku zadání, doplní jako mezerové (v šířce datového sloupce). Upravené hlavičky dosadí zpět do seznamu headerArrayList pro další zpracování (zařazení do výstupu a tisku).
- Je-li *headerArrayList* prázdný, zapíše jen jednu hlavičku s názvy sloupců z příkazu SELECT. Maximální šířky stanoví stejně jako u hlaviček z komentářových řádků.
- Seznam headerArrayList přepíše *původními uživatelskými hlavičkami* ze seznamu headerArrayListSaved pro případné nové spuštění metody *runScript()* z třídy *Q_PromptParameters* (i když jsou třeba oba seznamy prázdné).

Metoda *editNumericDecimal()* - formátování dekadických čísel

```
String editNumericDecimal(BigDecimal rs_DecimalValue, String colName)
```

- Metoda převádí parametr rs_DecimalValue typu BigDecimal podle lokalizačních pravidel a vrací editovaný řetězec decNumberEdited.
- Je-li seznam definic masek *patternArrayList neprázdný*, vyhledá v něm položku (řádek definice), se shodným jménem sloupce (převedeným do velkých písmen). Najde-li takovou položku, převezme z ní odpovídající masku. *Nenajde-li* žádnou takovou položku, převezme masku z parametrů aplikace (defaultDecPattern). Pak aplikuje masku na hodnotu sloupce (číslo typu BigDecimal):

```
NumberFormat nf = NumberFormat.getNumberInstance(currentLocale);
DecimalFormat df = (DecimalFormat) nf;
df.applyPattern(decimalPattern);
String decNumberEdited = df.format(rs_DecimalValue);
```

- Je-li seznam definic masek *patternArrayList prázdný* a je-li maska převzatá z parametrů aplikace *prázdná*, formátuje se číslo do znaků podle standardních lokalizačních pravidel.

```
NumberFormat nf = NumberFormat.getNumberInstance(currentLocale);
DecimalFormat df = (DecimalFormat) nf;
String decNumberEdited = df.format(rs_DecimalValue);
```

  Rozdíl proti přítomnosti masky je v tom, že *chybí* aplikace masky metodou *applyPattern()*.

- Je-li maska převzatá z parametrů aplikace *neprázdná*, aplikuje se na číslo stejně jako výše.

Metoda *processLevelBreaks()* - zpracování skupinových úrovní (L1, ...)

V cyklu přes sumarizované sloupce plní *koncové střádače* tj. střádače pro úroveň L0 (xxxTotalAccumulators) podle typu sloupce (dekadický, celočíselný, datum, čas, časové razítko a znakový) a způsobu střádání (SUM, AVG, MAX, MIN, COUNT).

V cyklu přes skupinové úrovně *pozpátku* (od poslední do nulté) zpracuje každou změnu skupiny.

- Porovná hodnotu sloupce uloženou v seznamu levelBreakValues s hodnotou právě přečtenou.
- Jestliže se nerovnají (došlo ke změně skupiny), zpracuje se daná úroveň, tj. sumarizační řádky dané úrovně: separační řádek s případnými proměnnými hodnotami a případné další řádky s úvodním textem a sumárními hodnotami (pro každý typ sumarizace nejvýše jeden). K tomu se volá metoda *summaryForLevels()*. V ní se dále volají metody

  getResetDecimalValue()
  getResetBinaryValue()
  getResetDateValue()
  getResetTimeValue()
  getResetTimestValue()
  getResetCharValue()

  které znovu dosadí počáteční hodnoty do střádačů a dále metoda

  adjustAndInsertToPrintLine() - vložení sumární hodnoty na správné místo v tiskovém řádku.

- Zapíše řádky do oblasti *resultTextArea* a právě přečtenou hodnotu sloupce zapíše do seznamu uložených hodnot *levelBreakValues* pro přístí porovnávání. K nastavení porovnávacích hodnot v seznamu levelBreakValues slouží dvě metody:

  getFirstLevelBreakValues() - nastaví počáteční hodnoty na začátku zpracování výsledné tabulky,
  getNextLevelBreakValues() - nastaví další hodnoty zde.

- Ve vnitřním cyklu přes sumarizované sloupce plní *skupinové střádače* tj. střádače pro úroveň **`L1`**, **`L2...`** (xxxAccumulators) podle typu sloupce (dekadický, celočíselný, datum, čas, časové razítko a znakový) a způsobu střádání (SUM, AVG, MAX, MIN, COUNT).
- Přečte jména a hodnoty všech sloupců *současného řádku* do seznamu detColNames a *detColValues* pro dosazení do proměnných (&sloupec) v separačním řádku úrovně.

Metoda *processLastLevelBreak()* - zpracování všech úrovní na konci výsledné tabulky

V cyklu přes skupinové úrovně pozpátku (od poslední do nulté) zpracuje každou změnu skupiny. Zde je *změna všech skupin* dána koncem výsledné tabulky (nikoliv změnou hodnoty sloupce).

- Zpracuje se daná úroveň, tj. sumarizační řádky dané úrovně: separační řádek s případnými proměnnými hodnotami a případné další řádky s úvodním textem a sumárními hodnotami (pro každý typ sumarizace nejvýše jeden) stejně jako u předchozích skupinových úrovní. K tomu se volá metoda
  summaryForTotals() a v ní metoda
  adjustAndInsertToPrintLine() - vložení sumární hodnoty na správné místo v tiskovém řádku.
- Řádky se zapíší do oblasti resultTextArea.

Je-li zadána úroveň **`L0`** (grand total), zpracuje se koncový sumář: separační řádek s případnými proměnnými hodnotami a případné další řádky s úvodním textem a sumárními hodnotami (pro každý typ sumarizace nejvýše jeden) stejně jako u předchozích skupinových úrovní. K tomu se volá metoda *summaryForGrandTotal()*.

Řádky se zapíší do oblasti resultTextArea.

Metoda *initializeAccumulators()* - inicializace úrovňových i koncových střádačů.

Úrovňových střádačů je tolik, kolik je skupinových úrovní (L1, L2, ...) krát počet sumarizovaných sloupců krát 5 (pro SUM, AVG, MAX, MIN, COUNT). Koncových střádačů je tolik, kolik je sumarizovaných sloupců krát 5 (pro SUM, AVG, MAX, MIN, COUNT).

Střádače se naplní počátečními hodnotami s použitím metod

getResetDecimalValue()
getResetBinaryValue()
getResetDateValue()
getResetTimeValue()
getResetTimestValue()
getResetCharValue()

Počáteční hodnoty se liší podle typu sloupce, přičemž u střádačů MAX a MIN jsou výjimky. Jsou to hodnoty sloupců přečtených z *prvního* řádku výsledné tabulky.

Obsluha zpráv (informačních a chybových)

Texty zpráv s doprovodnou informací jsou zapsány v poli ***retCode*** (dvojrozměrné textové pole). V první složce retCode[0] je uložen doprovodný text identifikující druh zprávy, v druhé složce retCode[1] je uložen text zprávy. Doprovodný text obsahuje údaje o tom, zda jde o dotaz “QUERY” nebo ne-dotaz “UPDATE”, a druh chyby “SQL_ERROR”, “PATTERN_ERROR”, “PARAMETER_ERROR”, “OTHER_ERROR”. Druh chyby se přidává k informaci o druhu skriptu (přiřetězí se), takže ostatní informace jsou zachovány. Ty se pak ve volajících programech testují příkazem

if (retCode[0].contains(“ERROR”) {...}

Zachytí-li *chybu* tato metoda (runScript()), zavolá metodu *reportError()*, která *připíše* text příkazu následovaný zprávou do oblasti *resultTextArea* a zavolá konstruktor *Q_PrintOneFile()*, který metodou *createWindow()* zobrazí text zprávy v okně. Po návratu z něj *připíše* obsah příkazu do textového souboru Print.txt v adresáři *workfiles* a do souboru v adresáři *printfiles* pod jménem skriptu (z proměnné *scriptName)* s koncovkou `.txt`.

Q_PromptParameters – zadání hodnot pro parametry příkazů SQL

Program je volán ze tříd *Q_ScriptRunCall* a *Q_ScriptEditCall* metodou *runPrompt()* se stejnými argumenty jako jsou argumenty volání metody *runScript()*, kterou také volá. Přebírá z ní *retCode[0]* (informaci "QUERY" nebo "UPDATE"), dosadí *retCode[1]* (chybovou zprávu nebo mezeru) a vrací *retCode* (dvousložkové pole).

Vytvoří okno s panelem nadpisů, s panelem seznamu parametrů SQL příkazu ve formě textových polí s popisy, s panelem tlačítek a s řádkem (JLabel) pro zprávy. Seznam parametrů vytváří zvlášť v metodě *buildInputPanel()*, protože je tvořen proměnlivým počtem řádků podle počtu parametrů z komentářových definic `--;?`. Textová pole jsou inicializována hodnotami z definic (4. údaj), jejich popisy tvoří texty z definic (3. údaj).

Pak zobrazí okno podle hodnoty *autoWindowSize* z parametrů aplikace. Je-li hodnota "Y", provede se metoda *pack()* a velikost okna se přizpůsobí obsahu, jinak bude okno zobrazeno v pevné velikosti.

Tlačítka umožňují následující funkce (lambda).

1. **return**Button.addActionListener(a -> {...});
   Tlačítko Končit. Smaže okno metodou dispose() a vrací se do volajícího programu.
2. **refreshOrig**Button.addActionListener(a -> {...});
   Tlačítko Zobrazit původní. Obnoví údaje v seznamu markerArrayList z předem uschovaných hodnot (savedMarkerArrayList), smaže okno (dataContentPane) a vytvoří je znovu metodou *buildDataPanel()*, čímž se obnoví původně zadané hodnoty parametrů příkazu. Pak zobrazí okno stejným způsobem jako na začátku.
3. **enter**Button.addActionListener(a -> {...});
   Tlačítko Potvrdit. Volá metodu enterData() a zobrazí okno jako v předchozích bodech buď s chybovou zprávou nebo bez ní.

Po stisku tlačítka *Končit* metoda *enterData()* kontroluje vstupní hodnoty textových polí ve všech řádcích parametrů. Ovšem jen tehdy, je-li v řádku zadán specifikovaný datový typ v řádku `--;?`. První chybu, na kterou narazí, zachytí a zobrazí v chybovém řádku:

retCode[1] = correctRow + parameterPosition + " (" + paramType + ")" + orOrig;
např. "Opravte řádek 1 (dekadické číslo) nebo zobrazte původní hodnoty".

Kontrola se provádí jen pro specifikované typy (zadaný text je převeden do velkých písmen): DEC, DECIMAL, NUMERIC, INT, INTEGER, BIGINT, DATE , TIME a TIMESTAMP. Po kontrole zavolá metodu *runScript()* třídy Q_ScriptRun se stejnými argumenty, které byly přijaty při volání této metody (runPrompt()). Volání metody runScript() je stejné jako z programu *Q_ScriptRunCall* (viz výše). Metoda *runScript()* provede SQL příkaz a vrátí návratový kód (String[] retCode). Byl-li SQL příkaz typu

“UPDATE”, dosadí do retCode[1] zprávu o provedení příkazu a vrátí hodnotu `false` (aby se ta zpráva zobrazila). Jinak vrátí hodnotu `true`.

Třída *EnterAction* umožňuje použít klávesu *Enter* se stejnou funkcí jako tlačítko *enterButton* (bod 4). Objekt této třídy je použit v sekvenci dvou příkazů

```
dataGlobalPanel.getInputMap(JComponent.WHEN_IN_FOCUSED_WINDOW).put(
        KeyStroke.getKeyStroke(“ENTER”), “enter”);
dataGlobalPanel.getActionMap().put(“enter”, new EnterAction());
```

které registrují tento objekt pro globální panel okna.

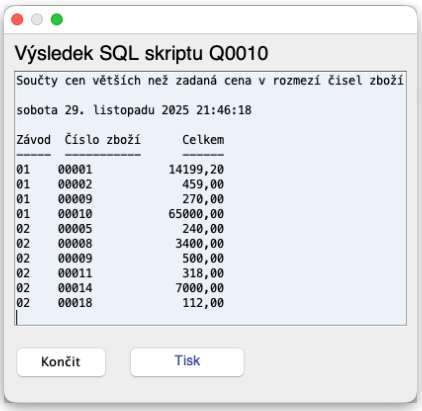
Q_PrintOneFile – zobrazení a tisk výsledku dotazu

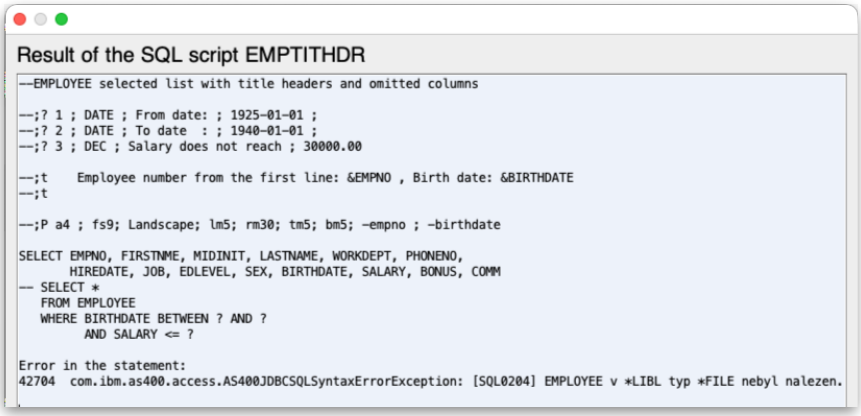
Je volán jen konstruktor (z programu *Q_ScriptRun*) s parametry

```
Q_PrintOneFile(String scriptName, JTextArea resultTextArea, int nbrHdrLines,
        ArrayList<String[]> headerArrayList, ArrayList<String[]> printArrayList,
        ArrayList<String>,  columnHeaders),
resultTextArea = výsledek dotazu nebo opis skriptu se zprávou,
nbrHdrLines = počet hlavičkových řádků (sloupcových hlaviček),
headerArrayList = seznam n-tic textů (sloupcových hlaviček),
printArrayList = seznam parametrů pro tisk (zpracuje se jen první řádek parametrů),
columnHeaders = seznam sloupcových hlavičkových řádků.
```

Zobrazení výsledku skriptu

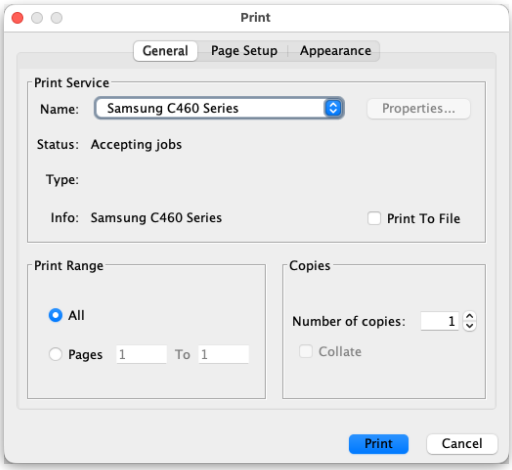
V konstruktoru se připraví objekty pro tisk a vyvolá se metoda *createWindow()*, která *zobrazí* výsledek SQL příkazu (oblasti *resultTextArea*) v okně. V případě chyby se na konci oblasti zapíše text zprávy.





1. **return**Button.addActionListener(a -> {...});
   Tlačítko Končit. Smaže okno metodou dispose() a vrací se do volajícího programu.
2. **print**Button.addActionListener(a -> {...});
   Tlačítko Tisk. Volá metodu ***printResult()*,** která vyvolá systémový dialog a tisk, pak smaže okno metodou dispose() a vytvoří nový objekt Q_PrintOneFile (čímž se oko znovu zobrazí). Metodu *printResult()* vyvolá také stisk klávesy *Enter*.

Tisk výsledku skriptu na papír



Metoda *printResult()* Vytvoří objekt *pageRenderer* třídy *FilePageRenderer*, který připraví, vykreslí a vytiskne stránku. Objekt umístí do vyčištěného objektu scrollPane a volá metodu validate(), aby se objekt mohl zobrazit. Dále do objektu *printerJob* třídy PrinterJob dosadí objekt pageRenderer a formát stránky pageFormat a spustí systémový tiskový dialog metodou *printerJob.printDialog(pageFormat)*, kde uživatel rozhodne, zda a jak se obsah vytiskne.

Konstruktor třídy *FilePageRenderer* přečte řádky textového souboru do seznamu *lineVector*, ten rozčlení do stránek (s očíslováním na konci stránky) podle vlastností objektu *pageFormat* pomocí metody *formatPages()*.

Metoda *formatPages()* předem vypočítá výšku a šířku tisknutelné oblasti na papíru podle předepsaných parametrů pro tisk (levý, pravý, horní, spodní okraj) v jednotkách tiskových bodů. Stanoví také celkový počet řádků na stránce a počet patičkových řádků (footer lines). Na druhé a další stránce tiskne sloupcové hlavičky (na první stránce již jsou zařazeny včetně názvu skriptu a data). Sloupcové hlavičky přebírá z parametru *columnHeaders* a *nbrHdrLines* (počet hlavičkových řádků). Hlavičky tiskne jednak při naplnění kapacity stránky a jednak při zjištění znaku **`\u0001`** na konci řádku. Tento znak byl přidán při zpracování řídicích skupin v progamu *Q_ScriptRun* v metodě processLevelBreaks() a po jejím provedení.

Stránky uloží do seznamů. Seznam *pageVector* obsahuje seznamy *page*, což jsou seznamy textových řádků.

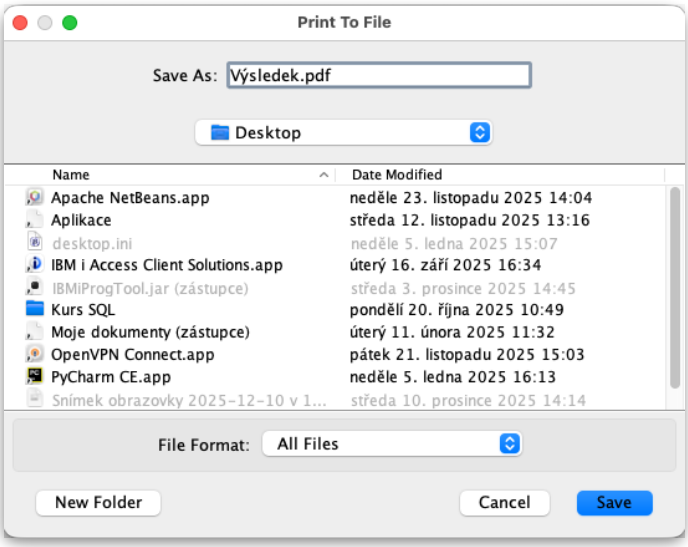
Třída *FilePageRenderer* dále obsahuje metodu

```
int print(Graphics g, PageFormat pageFormat, int pageIndex)
```

Metoda *print()* je implementací rozhraní *Printable* a spouští ji tiskový systém *automaticky* pro každou stránku zvlášť podle čísla pageIndex od 0 zvyšovaného po 1, jakmile je spuštěn tisk ze systémového dialogu vyvolaného metodou printerJob.pageDialog(). Metoda print() volá metodu paintComponent() nepřímo pomocí metody *paint()*. Metoda paintComponent() třídy JComponent vykreslí všechny řádky stránky do obdélníku dané velikosti podle zadaného písma, jeho velikosti a barvy. Metoda print() vrací tiskovému systému příznak, zda existuje další stránka podle čísla pageIndex: je-li číslo větší než počet stránek ve vektoru pageVector, vrátí NO_SUCH_PAGE a tisk končí. Jinak vrátí PAGE_EXISTS a tisk pokračuje s vyšším indexem.

Zápis výsledku skriptu do souboru

Zaškrtneme-li políčko *Print to file*, pak stiskem tlačítka *Print* místo tisku na papír vyvoláme nabídku adresářů.

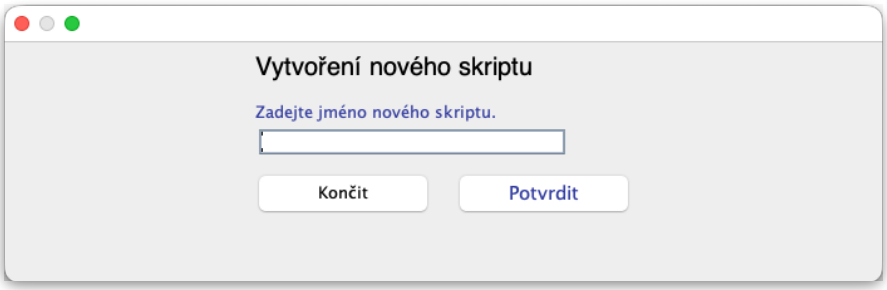


Z nabídky vybereme vhodný adresář, například `Desktop`, kam můžeme uložit výsledek flačítkem *Save*, třeba jako soubor `Výsledek.pdf`.

Q_PromptAddScript – zadání jména k vytvoření skriptu

Program je volán metodou z programu *Q_ScriptEditCall*. Zobrazí dialogové okno s textovým polem a dvěma tlačítky:

Končit (Exit), Potvrdit (Enter).

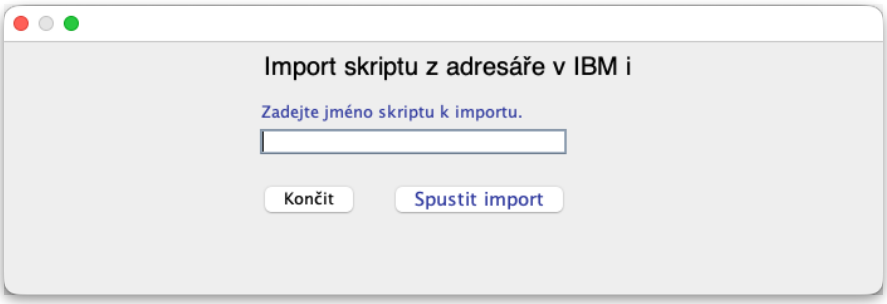


1. **return**Button.addActionListener(a -> {...});
   Tlačítko *Končit*. Nastaví příznak *canceled = true*, smaže okno metodou dispose() a vrací se do volajícího programu.
2. **enter**Button.addActionListener(a -> {...};
   Tlačítko *Spustit import*. Kontroluje, zda jméno skriptu není prázdné. Když není prázdné, nastaví příznak *canceled = false*, smaže okno metodou dispose() a vrací se do volajícího programu.

Stisk klávesy *Enter* provádí totéž co tlačítko *Potvrdit*.

Příznak *cancelled* se testuje ve volajícím programu *Q_ScriptEditCall*, jestli má (*false*) nebo nemá pokračovat ve výpočtu (*true*).

Q_PromptImportScript – zadání jména skriptu k importu z adresáře IFS



Program je volán metodou *runPrompt()* z programu *Q_ScriptEditCall*, není-li vybrán žádný řádek k *importu* skriptu z adresáře IFS v serveru. Zobrazí okno s textovým polem a tlačítky:

Končit (Exit), Zobrazit původní (Display original), Zobrazit vstup (Display input), Spustit import (Run import).

1. **return**Button.addActionListener(a -> {...});
   Tlačítko *Končit*. Smaže okno metodou dispose() a vrací se do volajícího programu.
2. **refreshOrig**Button.addActionListener(a -> {...});
   Tlačítko *Zobrazit původní*. Smaže obsah okna a znovu okno vytvoří a zobrazí.
3. **refreshEntered**Button.addActionListener(a -> {});
   Tlačítko *Zobrazit vstup*. Znovu zobrazí okno
4. **enter**Button.addActionListener(a -> {...};
   Tlačítko *Spustit import*. Kontroluje, zda jméno skriptu není prázdné. Když není prázdné, volá metodu *runImport()*.

Stisk klávesy *Enter* provádí totéž co tlačítko *Spustit import*.

Metoda *runImport()* přečte jméno souboru ze vstupního pole a volá přenos z IFS prostřednictvím třídy *Q_ImportOneFromAS400*, která je rozšířením třídy *javax.swing.SwingWorker*. Na objekt této třídy se spustí metoda *execute()*, která vyvolá metodu *doInBackground()*, která volá metodu *transferOneFromAS400()*.

Q_ImportOneFromAS400 – import vybraného skriptu z IFS

Program je volán ze třídy *Q_PromptForScriptName*, v metodě *runImport()*.

Třída *Q_ImportOneToAS400* je rozšířením třídy *javax.swing.SwingWorker*. Na objekt *importOne* této třídy se spustí paralelení proces metodou *execute()*, která vyvolá metodu *doInBackground()*, a ta volá metodu *transferOneFromAS400(selScriptFileName)*.

Paralelní proces je volen proto, aby při déletrvajícím pokusu o spojení se serverem mohla být v okně ihned zobrazena zpráva o probíhající akci.

- Metoda *transferOneFromAS400()* zjistí, zda lze připojit server metodou *AS400Ping#ping()* s časovým prodlením 8000 milisekund. Když *ping()* vrátí hodnotu *null*, vytvoří chybovou zprávu a také skončí vrácením hodnoty *null*.
- když *ping()* vyjde pozitivně, vytvoří objekt *as400Host* třídy *AS400* (IBM i Toolbox for Java), přičemž zobrazí přihlašovací okno, kde se uživatel musí identifikovat jménem a heslem.

Dále vytvoří objekt *as400Host* třídy AS400 (IBM i Toolbox for Java), přičemž zobrazí přihlašovací okno, kde se uživatel musí identifikovat jménem a heslem. Dále vytvoří objekt *ifsFile* třídy IFSFile s použitím cesty ke vstupnímu IFS souboru. Pak zkopíruje všechny

řádky ze vstupního IFS souboru do výstupního souboru v adresáři “scriptfiles” a vrátí zprávu informační nebo chybovou. Není-li vybrán žádný řádek, ohlásí chybovou zprávu.

Q_ExportOneToAS400 – export vybraného skriptu do IFS

Program je volán ze třídy *Q_ScriptEditCall*, z lambda funkce pro tlačítko *scriptListExportButton* (Save to server, Uložit do serveru).

Třída *Q_ExportOneToAS400* je rozšířením třídy *javax.swing.SwingWorker*. Na objekt *exportOne* této třídy se spustí paralelení proces metodou *execute()*, která vyvolá metodu *doInBackground()*, a ta volá metodu *transferOneToAS400(selScriptFileName)*.

Paralelní proces je volen proto, aby při déletrvajícím pokusu o spojení se serverem mohla být v okně ihned zobrazena zpráva o probíhající akci (“Transferring ...”).

- Metoda *transferOneToAS400()* zjistí, zda lze připojit server metodou *AS400Ping#ping()* s časovým prodlením 8000 milisekund. Když *ping()* vrátí hodnotu *null*, vytvoří chybovou zprávu a také skončí vrácením hodnoty *null*.
- když *ping()* vyjde pozitivně, vytvoří objekt *as400Host* třídy *AS400* (IBM i Toolbox for Java), přičemž zobrazí přihlašovací okno, kde se uživatel musí identifikovat jménem a heslem.

Dále vytvoří objekt *as400Host* třídy AS400 (IBM i Toolbox for Java), přičemž zobrazí přihlašovací okno, kde se uživatel musí identifikovat jménem a heslem. Dále vytvoří objekt *ifsFile* třídy IFSFile a vytvoří výstupní soubor v IFS adresáři, jestliže ještě neexistuje. Pak přenese skript z adresáře “scriptfiles” do adresáře IFS a vrátí zprávu informační nebo chybovou. Není-li vybrán žádný řádek, ohlásí chybovou zprávu.

Q_ImportAllFromAS400 – import všech skriptů z IFS

Program Q_ImportAllFromAS400 je realizován jako rozšíření třídy SwingWorker.

Program je volán z Q_Menu metodou *execute()*, která spustí paralelní proces a v něm metodu *doInBackground()* a v ní metodu *transferAllFromAS400()*.

Paralelní proces je volen proto, aby při déletrvajícím přenosu mohly být průběžně zobrazovány zprávy o importu jednotlivých souborů.

Metoda *transferAllFromAS400()*

- zjistí, zda lze připojit server metodou *AS400Ping#ping()* s časovým prodlením 8000 milisekund. Když *ping()* vrátí hodnotu *null*, vytvoří chybovou zprávu a také skončí vrácením hodnoty *null*.
- když *ping()* vyjde pozitivně, vytvoří objekt *as400Host* třídy *AS400* (IBM i Toolbox for Java), přičemž zobrazí přihlašovací okno, kde se uživatel musí identifikovat jménem a heslem.

Dále vytvoří objekt directory třídy IFSFile. Příkazem

```
IFSFile[] files = directory.listFiles();
```

vytvoří seznam souborů v IFS adresáři (directory) a jeden po druhém je kopíruje po řádcích do lokálních souborů v adresáři “scriptfiles”. Existující soubory přepíše, nové zapíše.

O přenosu každého souboru zapisuje zprávu do seznamu *messages* v panelu zpráv okna Q_Menu.

(Paralelní proces vnitřní třídy může v tomto případě modifikovat objekty - zprávy - třídy Q_Menu, odkud byl vyvolán.)

Q_ExportAllToAS400 – export všech skriptů do IFS

Program Q_ExportAllToAS400 je realizován jako rozšíření třídy SwingWorker.

Program je z Q_Menu volán metodou *execute()*, která spustí paralelní proces a v něm metodu *doInBackground()* a v ní metodu *transferAllToAS400()*.

Paralelní proces je volen proto, aby při déletrvajícím přenosu mohly být průběžně zobrazovány zprávy o exportu jednotlivých souborů.

- Metoda *transferAllToAS400()* zjistí, zda lze připojit server metodou *AS400Ping#ping()* s časovým prodlením 8000 milisekund. Když *ping()* vrátí hodnotu *null*, vytvoří chybovou zprávu a také skončí vrácením hodnoty *null*.
- když *ping()* vyjde pozitivně, vytvoří objekt *as400Host* třídy *AS400* (IBM i Toolbox for Java), přičemž zobrazí přihlašovací okno, kde se uživatel musí identifikovat jménem a heslem.

Dále vytvoří objekt as400Host třídy AS400 (IBM i Toolbox for Java), přičemž zobrazí přihlašovací okno, kde se uživatel musí identifikovat jménem a heslem. Pak probírá vstupní adresář “scriptfiles” a pro každý vstupní soubor vytvoří objekt *ifsFile* třídy IFSFile a z něj výstupní soubor v IFS adresáři, jestliže ještě neexistuje, a zkopíruje řádky vstupního souboru do výstupního IFS souboru.

O přenosu každého souboru zapisuje zprávu do seznamu *messages* v panelu zpráv okna Q_Menu.

(Paralelní proces vnitřní třídy může v tomto případě modifikovat objekty - zprávy - třídy Q_Menu, odkud byl vyvolán.)

Q_ConnectDB – spojení JDBC se serverem

Obsahuje metodu *connect()*, která vrací objekt connection typu Connection. Tento objekt představuje spojení s databází IBM DB2 for i.

Metoda *connect()*

- zjistí, zda lze připojit server metodou *AS400Ping#ping()* s časovým prodlením 8000 milisekund. Když *ping()* vrátí hodnotu *null*, vytvoří chybovou zprávu a také skončí vrácením hodnoty *null*.
- když *ping()* vyjde pozitivně, získá *JDBC ovladač* (driver) k databázi voláním

  Class.forName("com.ibm.as400.access.**AS400JDBCDriver**");

  a pak nastaví vlastnosti spojení (Properties conprop)

  "decimal separator"
  "sort language"

  z nichž jsou lokalizovány podle jazyka. Pak vytvoří objekt spojení

  connection = DriverManager.getConnection("jdbc:as400://" + host, conprop);

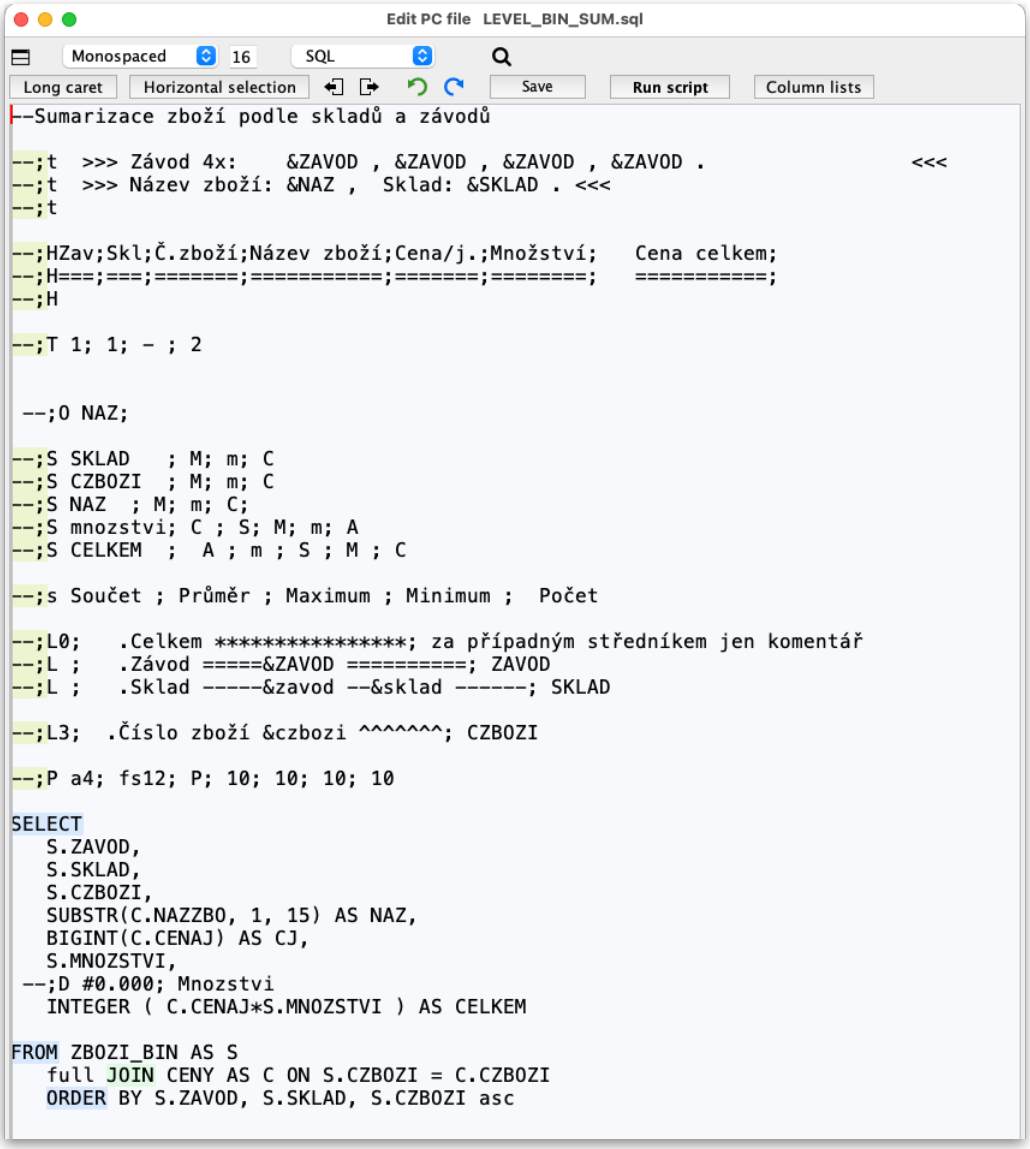
  nastaví automatické potvrzování transakcí

  connection.set**AutoCommit(true)**;

  a vrátí objekt *connection* nebo v případě chyby vrátí hodnotu *null*.

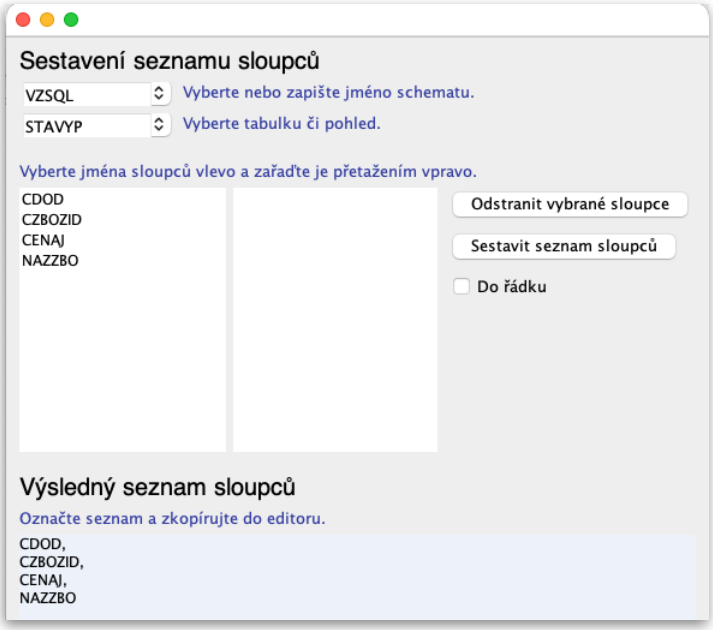
Poznámka: Program obsahuje též metodu *disconnect()*, ale ta se nepoužívá.

Q_ScriptEdit – editor skriptu



Program je převzatá a upravená třída *EditFile* z aplikace *IBMiProgTool*. Program je volán z programu *Q_ScriptEditCall* voláním konstruktoru *Q_ScriptEdit()*.

1. **save**Button.addActionListener(saveAction);
   Tlačítko Save nebo klávesa Ctrl-S. *SaveAction* je třída, kde se volá metoda *rewritePcFile*(), která uloží obsah *textArea* do odpovídajícího souboru skriptu v adresáři *scriptfiles*.
2. **runScript**Button.addActionListener(ae -> {...};
   Tlačítko Run script. Volá se metoda *rewritePcFile*(), jako u Save, přečte se uložený soubor a vyvolá se metoda *performScript()* třídy *Q_Script**Run**Call*. Zprávu o výsledku pošle do objektu třídy *Q_Script**Edit**Call*.
3. **columnList**Button.addActionListener(ae -> {...};
   Tlačítko Column lists. Vyvolá metodu *createWindow()* třídy *Q_ColumnLists*, která vrací výsledek *retCode*. Chybovou zprávu posílá do objektu třídy *Q_Script**Edit**Call*.

   retCode je pole o dvou prvcích, kde první prvek je text, který obsahuje buď kód "ERROR" nebo text bez tohoto kódu. Druhý prvek je text zprávy.

Q_ColumnLists – sestavení seznamu sloupců